



Proyecto Final

Profesora: Jessica Santizo Galicia

Ayudante de Teoría: Diego Antonio Villalba González

Ayudante de Laboratorio: Ares Gael Castro Romero

Descripción general. El proyecto integrador tiene como objetivo que cada equipo aplique, de forma cohesionada, los principales temas del curso sobre un problema real y relevante de México. La solución deberá cubrir todas las etapas del ciclo de vida de un proyecto de minería de datos: definición del problema, obtención y exploración de datos, modelado supervisado y no supervisado, implementación técnica con buenas prácticas de ingeniería de software, y comunicación formal de resultados.

Tema, problema y justificación

Cada equipo seleccionará libremente su tema de **una problemática en la sociedad mexicana** dentro de alguna de las siguientes áreas (no excluyentes):

- Seguridad pública
- Salud pública
- Educación
- Medio ambiente
- Agricultura y alimentación
- Economía y finanzas
- Transporte y movilidad
- Turismo y cultura
- Vivienda y urbanismo
- Energía
- Comercio e industria

La justificación del tema deberá responder explícitamente a las siguientes cuatro preguntas:

1. **¿Cuál es el problema?** Descripción concreta y acotada del fenómeno que se analiza.
2. **¿Por qué es relevante?** Impacto social, económico o ambiental; evidencia de que el problema existe y es significativo.
3. **¿Qué se busca lograr?** Objetivo claro y medible del análisis (p. ej., predecir, segmentar, identificar patrones).
4. **¿Por qué este *dataset*?** Pertinencia de los datos elegidos para responder el objetivo planteado.

Datos (*Dataset*)

Requisitos mínimos obligatorios

- Datos reales relacionados con México (no generados artificialmente).
- **Mínimo 100 000 registros** (filas) y **al menos 8 variables relevantes** (columnas).
- Presencia de variables **numéricas y categóricas**.
- Fuente verificable, pública y citable (URL directa o DOI).
- Los datos deben permitir **tanto** modelado supervisado **como** agrupamiento.

Restricciones. No se aceptarán datasets generados artificialmente, aumentados por duplicación o síntesis de registros, ni datasets de uso privado sin fuente verificable. Los datos deben poder ser descargados o accedidos por el equipo docente para verificación.

Fuentes sugeridas

Se recomienda utilizar fuentes con datos abiertos verificables y descargables, por ejemplo: [INEGI Datos Abiertos](#), [Plataforma Nacional de Datos Abiertos](#), [CONEVAL](#), [Dirección General de Epidemiología](#), [SESNP](#), [CONAGUA Datos Abiertos](#), [Sistema de Información Hidrológica](#), [Data México](#), [DENUE](#), [CONABIO Geoportal](#), [API Banxico](#) y [Datos Abiertos CDMX](#).

Análisis Exploratorio de Datos (EDA)

El EDA es la primera etapa analítica y deberá realizarse **antes** de cualquier modelado. Su propósito es comprender la estructura, calidad y distribución de los datos, e informar las decisiones de preprocesamiento y modelado posteriores.

El EDA deberá incluir, como mínimo:

1. **Descripción general:** dimensiones del dataset (filas $\times$ columnas), tipos de datos por variable, fuente y fecha de extracción.
2. **Calidad de datos:** conteo de valores faltantes por columna (y porcentaje), registros duplicados, y estrategia de manejo elegida con justificación.
3. **Estadísticas descriptivas:** media, mediana, desviación estándar, cuartiles y rango para variables numéricas; frecuencias y moda para variables categóricas.
4. **Detección de valores atípicos:** uso de boxplots, regla IQR o z-score; decisión justificada sobre su tratamiento.
5. **Distribuciones:** histogramas de todas las variables numéricas relevantes con interpretación.
6. **Relaciones entre variables:** matriz de correlaciones (Pearson o Spearman) con heatmap; scatter plots de pares relevantes.
7. **Análisis de variables categóricas:** gráficas de barras con frecuencias; análisis de balance de clases si aplica al modelado supervisado.
8. **Visualizaciones adicionales** (si aplica): mapas geográficos, series de tiempo u otras gráficas pertinentes al dominio del problema.

Importante. Cada visualización debe ir acompañada de una interpretación escrita que explique qué se observa y qué implicación tiene para el análisis. No basta con mostrar la gráfica.

Modelo supervisado

Qué se debe construir

Deberán entrenar **al menos un** modelo de aprendizaje supervisado. El tipo de tarea (clasificación o regresión) deberá derivarse directamente del objetivo del proyecto.

Algoritmos sugeridos (no exclusivos): Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors, Gradient Boosting, XGBoost, Support Vector Machine, Linear/Ridge/Lasso Regression.

Proceso requerido

1. **Definición de la variable objetivo:** indicar qué se predice y por qué esa variable es la más adecuada para el objetivo del proyecto.
2. **Preprocesamiento específico para el modelo:** codificación de variables categóricas, normalización o estandarización de variables numéricas, manejo de desbalance de clases si aplica.
3. **Selección de variables de entrada:** justificar qué variables se incluyen y cuáles se excluyen (con base en el EDA, dominio del problema o técnicas de selección de características).
4. **División de datos:** separación en conjunto de entrenamiento y prueba (mínimo 70/30 o 80/20), con semilla aleatoria fija para reproducibilidad.
5. **Entrenamiento y ajuste:** ajuste de hiperparámetros con validación cruzada o búsqueda en cuadrícula, documentando la configuración final.
6. **Evaluación e interpretación:** métricas completas según la tarea (ver abajo) e interpretación de resultados en el contexto del problema real.
7. **Comparación con línea base:** el modelo debe superar un clasificador/regresor trivial (p.ej., clasificador mayoritario o media del objetivo); de no superarlo, se deberá analizar el por qué.

Métricas de evaluación

Tarea	Métricas requeridas
Clasificación	Accuracy, Precision, Recall, F1-score (macro y ponderado), matriz de confusión, curva ROC y AUC si aplica.
Regresión	MAE, MSE, RMSE, R^2 (coeficiente de determinación), gráfica de residuales.

Persistencia del modelo

El modelo entrenado deberá guardarse en disco usando `pickle` o `joblib` y cargarse correctamente en una demostración separada del entrenamiento. La persistencia es un requisito técnico evaluado.

Agrupamiento (*Clustering*)

Qué se debe construir

Deberán aplicar **obligatoriamente** al menos una técnica de agrupamiento no supervisado. El agrupamiento debe tener un propósito analítico claro: identificar segmentos, perfiles o patrones ocultos en los datos que complementen o enriquezcan los hallazgos del modelo supervisado.

Algoritmos sugeridos: K-Means, DBSCAN, Agrupamiento jerárquico (Ward, complete, average), Gaussian Mixture Models.

Proceso requerido

1. **Selección y justificación de variables:** indicar qué variables se usan para el agrupamiento y por qué (pueden diferir de las del modelo supervisado).
2. **Preprocesamiento:** normalización de variables (obligatorio para K-Means y KDE); reducción de dimensionalidad con PCA si se trabaja con muchas variables.
3. **Selección del algoritmo y justificación:** por qué ese algoritmo es adecuado para los datos (considerar forma de los grupos, escala, ruido).

4. **Elección del número de grupos** (si el algoritmo lo requiere): uso de al menos uno de los siguientes criterios: método del codo (inercia), coeficiente de silueta, dendrograma.
5. **Visualización de grupos**: scatter plot con colores por cluster (con PCA o t-SNE para reducción si hay más de 2 dimensiones).
6. **Interpretación de perfiles**: descripción de cada grupo con base en las variables originales (estadísticas por cluster); los grupos deben tener un nombre o etiqueta interpretativa, no solo un número.

Requerimientos técnicos

Lenguaje y librerías

Python es el lenguaje obligatorio. Se recomienda el uso de: **pandas**, **numpy**, **scikit-learn**, **matplotlib**, **seaborn**, **plotly** (para visualizaciones interactivas en Quarto), y **xgboost** o **lightgbm** opcionalmente.

Programación Orientada a Objetos (POO)

El código fuente deberá estar organizado usando clases y objetos. No se acepta un único script lineal o un notebook sin estructura. Se evaluará:

- Encapsulamiento de responsabilidades en clases coherentes.
- Uso de herencia o composición donde sea natural.
- Separación entre lógica de datos, modelado y reporte.

Patrón de diseño

Deberá implementarse **al menos un** patrón de diseño de software, elegido entre los siguientes, y justificar su elección en el reporte:

Patrón	Cuándo es adecuado
Pipeline	Cuando el procesamiento de datos ocurre en etapas secuenciales bien definidas (ej., limpieza → encoding → escalado → modelo).
Factory Method	Cuando se necesita crear instancias de diferentes modelos o algoritmos de forma intercambiable sin cambiar el código cliente.
Strategy	Cuando se desea intercambiar algoritmos de preprocesamiento, modelado o evaluación en tiempo de ejecución.
Repository	Cuando se desea abstraer el acceso a datos (CSV, base de datos, API) detrás de una interfaz uniforme.

Reproducibilidad y control de versiones

- Todas las semillas aleatorias (**random_state**) deben fijarse explícitamente.
- El repositorio en **GitHub** debe tener historial de commits que evidencie la participación de todos los integrantes.
- Archivo **requirements.txt** con todas las dependencias y sus versiones exactas (generado con **pip freeze** o equivalente).
- Archivo **.gitignore** adecuado (excluir **__pycache__**, **.ipynb_checkpoints**, datos grandes, etc.).

Documentación con Quarto y GitHub Pages

El proyecto deberá incluir **documentación técnica interactiva** generada con Quarto (**.qmd**), compilada como sitio HTML estático en la carpeta **docs/** y desplegada públicamente con **GitHub Pages**.

Contenido mínimo del sitio

1. Descripción del problema y del dataset.
2. EDA con visualizaciones (estáticas o interactivas con Plotly).
3. Resultados del modelo supervisado (métricas y gráficas).
4. Resultados del clustering (visualización e interpretación de grupos).
5. Instrucciones de instalación y ejecución del proyecto.

Configuración mínima

- Archivo `_quarto.yml` en la raíz del repositorio con `output-dir: docs`.
- GitHub Pages configurado en: *Settings* → *Pages* → *Branch: main / docs*.
- URL del sitio desplegado incluida en el `README.md`.

Entregables

Fecha límite: todos los entregables deben estar disponibles en el repositorio de GitHub **antes** de la presentación oral del equipo el jueves 10 de junio.

#	Entregable	Descripción / ubicación esperada
1	Dataset o enlace	Datos en <code>data/</code> o enlace directo a fuente oficial en el <code>README.md</code>
2	Notebooks	Notebooks organizados en <code>notebooks/</code> (EDA, modelado, clustering)
3	Código fuente	Clases y módulos Python en <code>src/</code>
4	Modelos serializados	Archivos <code>.pkl</code> o <code>.joblib</code> en <code>models/</code>
5	Documentación Quarto	Sitio HTML en <code>docs/</code> desplegado en GitHub Pages
6	Reporte final	PDF en <code>reports/</code> (ver Sección 9)
7	Diagrama UML	Diagrama de arquitectura en <code>diagrams/</code>
8	Presentación (slides)	Archivo de slides en <code>reports/</code>
9	Video de exposición	Enlace al video (YouTube, Drive, etc.) en el <code>README.md</code> ; máx. 15 min
10	<code>README.md</code>	Instrucciones de instalación, ejecución y URL del sitio Quarto
11	<code>requirements.txt</code>	Dependencias con versiones exactas

Estructura sugerida del repositorio

```
Proyecto_Mineria_Datos/  
|-- data/           # Datos crudos y procesados (o enlace a fuente)  
|-- notebooks/      # Notebooks de EDA, modelado y clustering  
|-- src/            # Código fuente Python (clases, módulos)  
|-- models/         # Modelos serializados (.pkl / .joblib)  
|-- reports/        # Reporte PDF y slides de presentación  
|-- diagrams/       # Diagrama UML de la arquitectura  
|-- docs/           # Sitio Quarto compilado (GitHub Pages)  
|-- _quarto.yml      # Configuración de Quarto  
|-- README.md  
|-- requirements.txt  
'-- .gitignore
```

Reporte final

El reporte es un documento técnico formal en PDF. Deberá ser autocontenido: un lector que no haya visto el código ni los datos debe poder entender completamente el trabajo realizado.

Secciones obligatorias:

1. **Portada:** nombre del proyecto, integrantes, número de cuenta, fecha.
2. **Introducción:** problema, objetivo, justificación y valor del proyecto.
3. **Descripción del dataset:** fuente, variables (con definición de cada una), tamaño, y limitaciones conocidas de los datos.
4. **Limpieza y preparación de datos:** decisiones tomadas sobre valores faltantes, duplicados, outliers y transformaciones; todo con justificación.
5. **Análisis Exploratorio:** visualizaciones con interpretación narrativa.
6. **Modelo supervisado:** descripción del proceso, resultados, métricas y análisis crítico de los resultados.
7. **Clustering:** descripción del proceso, criterio de selección del número de grupos, visualización e interpretación de perfiles.
8. **Arquitectura y diseño:** patrón de diseño implementado con justificación; diagrama UML de la arquitectura.
9. **Reutilización del modelo:** demostrar que el modelo guardado puede cargarse y usarse para hacer predicciones nuevas sin re-entrenar.
10. **Conclusiones:** hallazgos principales, limitaciones del análisis y trabajo futuro.
11. **Referencias:** formato APA o IEEE, mínimo 5 referencias (incluyendo fuente del dataset y literatura técnica).

Presentación oral

La exposición se realizará el **jueves 10 de junio** a partir de las **8:00 am** en el salón de clase.

- Duración máxima: **15 minutos** de presentación + preguntas del equipo docente.
- **Todos los integrantes** deberán estar presentes y participar activamente; la asignación de secciones a cada integrante es libre, pero debe ser equitativa.
- Se realizará una **demostración en vivo**: cargar el modelo guardado y ejecutar una predicción nueva en tiempo real.

Temas que deberá cubrir la presentación:

1. Problema y dataset (motivación, fuente, características).
2. Hallazgos del EDA (máximo 3 visualizaciones clave con interpretación).
3. Modelo supervisado: proceso, métricas y conclusión sobre su desempeño.
4. Clustering: algoritmo elegido, número de grupos y descripción de cada perfil.

5. Patrón de diseño: cuál, por qué y cómo se implementó.
6. Demostración en vivo de la reutilización del modelo.
7. Conclusiones y limitaciones.

Modalidad y organización del equipo

- El proyecto puede realizarse de forma **individual o en equipos de hasta 4 integrantes**.
- Deberá designarse un **líder del proyecto** responsable de la coordinación y comunicación con el equipo docente.
- El historial de commits del repositorio debe evidenciar la **contribución de todos los integrantes**; un equipo con commits de un solo miembro será penalizado.

Evaluación

Peso total: este proyecto representa el **30 %** de la calificación final del curso, distribuido en tres rubros como se detalla a continuación.

Rubro	Criterios evaluados	Peso
Código y desarrollo técnico	Organización y calidad del código; implementación de POO; patrón de diseño justificado; modelo supervisado funcional y evaluado; clustering funcional e interpretado; persistencia y reutilización correcta del modelo; reproducibilidad del pipeline (semillas fijas, <code>requirements.txt</code>).	15 %
Reporte final	Claridad y rigor en el planteamiento del problema; justificación del dataset; profundidad del EDA; análisis crítico de resultados (modelo y clustering); conclusiones fundamentadas; redacción y formato del documento.	10 %
Exposición oral	Claridad de la presentación; demostración en vivo exitosa; capacidad de respuesta a preguntas técnicas; participación equilibrada del equipo.	5 %
Total		30 %

Causas de penalización automática: (1) uso de dataset generado artificialmente; (2) historial de commits que evidencie contribución de un solo integrante; (3) código no reproducible (sin semillas fijas o sin `requirements.txt`); (4) modelo guardado que no pueda cargarse y ejecutarse independientemente del notebook de entrenamiento; (5) entregables subidos después de la presentación oral.